

WEEK 7 Practical OSSP

PART 1 — Program to display memory-segment addresses

```
ashborn@DESKTOP-I4G5Q4N:~$ mkdir week7_p
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/week7_p
ashborn@DESKTOP-I4G5Q4N:~/week7_p$ pwd
/home/ashborn/week7_p
ashborn@DESKTOP-I4G5Q4N:~/week7_p$ nano vm_demo.c
ashborn@DESKTOP-I4G5Q4N:~/week7_p$ ./vm_demo
-bash: ./vm_demo: No such file or directory
ashborn@DESKTOP-I4G5Q4N:~/week7_p$ gcc -O0 -g vm_demo.c -o vm_demo
ashborn@DESKTOP-I4G5Q4N:~/week7_p$ ./vm_demo
PID: 69

Code address      : 0x7f91c59da209
Global variable   : 0x7f91c59dd010
Static global     : 0x7f91c59dd014
Static local      : 0x7f91c59dd018
Heap variable     : 0x7fffd26992a0
Stack variable    : 0x7fffd26423c

Press Enter to terminate...
```

Observation: Created the C program in a new folder and compiled it. The important observation is that different types of variables are stored in different regions of the process's virtual address space.

PART 2 — Examine the process using /proc/<PID>/maps

1. Terminal 1 and Terminal 2

```
ashborn@DESKTOP-I4G5Q4N:~/week7_p$ gcc -O0 -g vm_demo.c -o vm_demo
ashborn@DESKTOP-I4G5Q4N:~/week7_p$ ./vm_demo
PID: 69

Code address      : 0x7f91c59da209
Global variable   : 0x7f91c59dd010
Static global     : 0x7f91c59dd014
Static local      : 0x7f91c59dd018
Heap variable     : 0x7fffd26992a0
Stack variable    : 0x7fffd26423c

Press Enter to terminate...
```

```
ashborn@DESKTOP-I4G5Q4N:~$ cat /proc/69/maps
7f91c5700000-7f91c5701000 rw-p 00000000 00:00 0
7f91c5770000-7f91c5798000 r--p 00000000 00:00 991594 /usr/lib/x86_64-
7f91c5798000-7f91c5920000 r-xp 00020000 00:00 991594 /usr/lib/x86_64-
7f91c5920000-7f91c596f000 r--p 001b0000 00:00 991594 /usr/lib/x86_64-
7f91c596f000-7f91c5973000 r--p 001fe000 00:00 991594 /usr/lib/x86_64-
7f91c5973000-7f91c5975000 rw-p 00202000 00:00 991594 /usr/lib/x86_64-
7f91c5975000-7f91c5982000 rw-p 00000000 00:00 0
7f91c5982000-7f91c5991000 r--p 00000000 00:00 991591 /usr/lib/x86_64-
7f91c5991000-7f91c599b000 r-xp 00001000 00:00 991591 /usr/lib/x86_64-
7f91c599b000-7f91c599c000 r-xp 00020000 00:00 991591 /usr/lib/x86_64-
7f91c599c000-7f91c59c5000 r--p 0002c000 00:00 991591 /usr/lib/x86_64-
7f91c59c5000-7f91c59c6000 r--p 00031000 00:00 991591 /usr/lib/x86_64-
7f91c59c6000-7f91c59c8000 r--p 00036000 00:00 991591 /usr/lib/x86_64-
7f91c59c8000-7f91c59ca000 rw-p 00030000 00:00 991591 /usr/lib/x86_64-
7f91c59ca000-7f91c59d2000 rw-p 00000000 00:00 0
7f91c59d2000-7f91c59da000 r--p 00000000 00:00 549528 /home/ashborn/we
7f91c59da000-7f91c59db000 r-xp 00001000 00:00 549528 /home/ashborn/we
7f91c59db000-7f91c59dd000 r--p 00001000 00:00 549528 /home/ashborn/we
7f91c59dd000-7f91c59de000 r--p 00001000 00:00 549528 /home/ashborn/we
7f91c59de000-7f91c59df000 rw-p 00003000 00:00 549528 [heap]
7fffd2690000-7fffd269a000 rw-p 00000000 00:00 0
7fffd269a000-7fffd269e000 rw-p 00000000 00:00 0 [stack]
7fffd269e000-7fffd269f000 r-xp 00000000 00:00 0 [vdso]
ashborn@DESKTOP-I4G5Q4N:~$
```

2. Make the /proc/maps output easier to understand

```
ashborn@DESKTOP-I4G5Q4N:~$ grep -E 'vm_demo|\\[heap\\]|\\[stack\\]' /proc/69/maps
7f91c59d9000-7f91c59da000 r--p 00000000 00:00 549528 /home/ashborn/week7_p/vm_demo
7f91c59da000-7f91c59db000 r-xp 00001000 00:00 549528 /home/ashborn/week7_p/vm_demo
7f91c59db000-7f91c59dc000 r--p 00002000 00:00 549528 /home/ashborn/week7_p/vm_demo
7f91c59dc000-7f91c59dd000 r--p 00002000 00:00 549528 /home/ashborn/week7_p/vm_demo
7f91c59dd000-7f91c59de000 rw-p 00003000 00:00 549528 /home/ashborn/week7_p/vm_demo
7ffffd2699000-7ffffd26ba000 rw-p 00000000 00:00 0 [heap]
7ffffd9a66000-7ffffda266000 rw-p 00000000 00:00 0 [stack]
ashborn@DESKTOP-I4G5Q4N:~$
```

Summary:

A C program was developed to display the virtual addresses of code, global, static, heap and stack variables. The program also displayed its process ID so that its memory layout could be examined through the Linux /proc filesystem.

The /proc/<PID>/maps file was used to observe the virtual memory regions allocated to the running process. The mappings showed separate regions for the executable, heap, stack and shared libraries, along with their access permissions.

PART 3 — Memory Analysis with pmap

```
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/week7_p
ashborn@DESKTOP-I4G5Q4N:~/week7_p$ ./vm_demo
PID: 27
Code address      : 0x7fc751592009
Global variable   : 0x7fc75159c010
Static global     : 0x7fc75159c014
Static local      : 0x7fc75159c018
Heap variable     : 0x7ffffd9272a0
Stack variable    : 0x7ffffe829dd4c
Press Enter to terminate...
```

```
ashborn@DESKTOP-I4G5Q4N:~$ pmap -x 27
27:  ./vm_demo
Address      Kbytes      RSS Dirty Mode Mapping
-----
total kB      10720      0      0
ashborn@DESKTOP-I4G5Q4N:~$ pmap -x 27 | grep -E 'vm_demo|heap|stack|total'
27:  ./vm_demo
total kB      10720      0      0
ashborn@DESKTOP-I4G5Q4N:~$
```

PART 4 — Check Virtual Memory Statistics

```
ashborn@DESKTOP-I4G5Q4N:~$ cat /proc/27/status | grep -E 'VmPeak|VmSize|VmRSS|VmData|VmStk|VmExe|VmLib'
VmPeak: 0 kB
VmSize: 10720 kB
VmRSS: 648 kB
VmData: 0 kB
VmStk: 0 kB
VmExe: 4 kB
VmLib: 0 kB
```

PART 5 — Observe the Page Size

```
ashborn@DESKTOP-I4G5Q4N:~$ getconf PAGESIZE  
4096
```

Conclusion:

The program successfully displayed the virtual addresses of code, global, static, heap and stack variables. The `/proc/<PID>/maps` file confirmed that these regions are mapped separately within the process's virtual address space.

Using `pmap` and `/proc/<PID>/status`, the memory organization and memory usage of the running process were analyzed. The experiment also demonstrated the role of virtual memory, page tables and the TLB in translating virtual addresses to physical addresses.